

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Disciplina: Teste de Software

Refatoração de Testes e Detecção de Test Smells

Aluna: **Gabriela Alvarenga Cardoso**

Matrícula: **844469**

Professor: Cleiton Tavares

Repositório: <https://github.com/gabialvarenga/test-smelly>

Outubro de 2026

1. Análise de Smells

Todos os testes de `test/userService.smelly.test.js` passam, mas a suíte esconde problemas que reduzem a sua capacidade de encontrar defeitos. Os três principais estão descritos abaixo.

1.1 Lógica Condicional no Teste (Conditional Test Logic)

Onde: "deve desativar usuários se eles não forem administradores", que percorre uma lista com `for` e decide por `if/else` quais `expect` serão executados.

Por que é um mau cheiro: o teste ganha caminhos próprios e deixa de ser uma especificação direta. Para saber o que ele verifica é preciso simular a execução mentalmente.

Risco: se os dados mudarem (por exemplo, o objeto deixar de trazer `isAdmin`), um ramo inteiro de asserções deixa de rodar sem nenhum aviso. Além disso, uma falha na primeira iteração esconde o resultado da segunda.

1.2 Teste que Passa Silenciosamente (exceção em `try/catch`)

Onde: "deve falhar ao criar usuário menor de idade", em que o único `expect` está dentro do `catch`.

Por que é um mau cheiro: o teste só faz alguma verificação se a exceção acontecer. Se `createUser` não lançar erro, o `catch` não roda e o teste termina sem nenhuma asserção.

Risco: falso positivo. Removendo a validação `idade < 18`, o teste continua verde (comprovado na Figura 4). É o mesmo "mutante sobrevivente" estudado no trabalho de teste de mutação.

1.3 Teste Frágil (Fragile Test)

Onde: "deve gerar um relatório de usuários formatado", que exige a linha exata `ID: ${id}, Nome: Alice, Status: ativo\n` e o início `--- Relatório de Usuários ---`.

Por que é um mau cheiro: o teste fica acoplado à formatação do texto, e não ao conteúdo que importa. O próprio código de produção avisa que o formato do relatório pode mudar.

Risco: qualquer ajuste de espaço, ordem ou pontuação quebra o teste sem haver bug. Alarmes falsos frequentes fazem a equipe perder a confiança na suíte.

Outros achados: o teste "criar e buscar" é um *Eager Test*: valida a criação e a consulta de usuário de uma só vez, então, se falhar, não fica claro qual das duas operações tem defeito. Há também um `test.skip` vazio que dá a falsa impressão de que o cenário de lista vazia está coberto.

2. Processo de Refatoração

Escolhi o teste de desativação, o mais problemático da suíte: tem lógica condicional, mistura dois comportamentos e concentra 3 dos 4 erros do ESLint.

Antes (`userService.smelly.test.js`)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (userService.clean.test.js)

```
describe('deactivateUser', () => {
  test('deve retornar true ao desativar um usuário comum', () => {
    const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

    const resultado = userService.deactivateUser(usuarioComum.id);

    expect(resultado).toBe(true);
  });

  test('deve marcar o usuário comum como inativo após desativá-lo', () => {
    const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

    userService.deactivateUser(usuarioComum.id);

    const usuarioAtualizado = userService.getUserById(usuarioComum.id);
    expect(usuarioAtualizado.status).toBe('inativo');
  });

  test('deve retornar false ao tentar desativar um administrador', () => { /* mesmo formato */ });

  test('deve manter o administrador ativo após tentativa de desativação', () => { /* mesmo formato */ });
});
```

Decisões tomadas

- **Sem for e sem if:** cada cenário (usuário comum e administrador) virou um teste próprio. Todo expect sempre executa, o que elimina a lógica condicional e os erros jest/no-conditional-expect.
- **Padrão AAA:** cada teste cria só o usuário de que precisa (Arrange), chama deactivateUser uma vez (Act) e verifica um único comportamento (Assert), com as fases separadas por linhas em branco.
- **Retorno e efeito separados:** o valor devolvido por deactivateUser e a alteração de status ficam em testes diferentes. Se só um deles quebrar, o Jest mostra qual, sem que eu precise depurar.
- **Cobertura da regra de negócio do admin:** além do valor retornado, criei um teste que consulta o administrador depois da tentativa e confere que ele segue ativo. Assim, a regra "admins não podem ser desativados" fica protegida pelo estado do usuário, e não só pela resposta do método.
- **Nomes que funcionam como documentação:** ao ler o relatório do Jest, cada linha diz em português o que o sistema deveria fazer (por exemplo, "deve manter o administrador ativo após tentativa de desativação"), sem precisar abrir o código.

O mesmo critério foi aplicado ao resto da suíte: o try/catch virou expect(fn).toThrow(...); o relatório passou a ser verificado por conteúdo (toContain e stringContaining); "criar e buscar" foi dividido em testes de createUser e getUserById; o test.skip foi implementado; e foram adicionados casos que faltavam (campos obrigatórios, id inexistente). O arquivo novo ficou organizado em um bloco describe por método do serviço, somando 14 casos.

3. Relatório da Ferramenta

O ESLint 8 foi configurado com o .eslintrc.json do enunciado (eslint:recommended, plugin:jest/recommended e as regras jest/no-disabled-tests, jest/no-conditional-expect e jest/no-identical-title).

```
Windows PowerShell — test-smelly-fork

PS C:\...\test-smelly-fork> npx eslint .
... \test-smelly-fork\test\userService.smelly.test.js
44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
77:3  warning Tests should not be skipped    jest/no-disabled-tests
77:3  warning Test has no assertions         jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
PS C:\...\test-smelly-fork> █
```

Figura 1 – Lint da suíte original: quatro expect condicionais (erros) e o teste pulado sem asserção (avisos).

Linha	Regra	Smell correspondente
44, 46, 49	jest/no-conditional-expect	Lógica condicional (for + if/else)
73	jest/no-conditional-expect	expect dentro do catch
77	jest/no-disabled-tests	Teste desabilitado (test.skip)
77	jest/expect-expect	Teste sem asserções

Como a ferramenta automatizou a detecção: em segundos, e sem depender de leitura atenta, o ESLint apontou a linha exata de cada expect condicional e do teste desabilitado, cobrindo os smells 1.1 e 1.2 da análise manual. Por outro lado, ele não detectou o Teste Frágil nem o Eager Test: esses smells dependem de entender a intenção do teste e só aparecem na revisão humana. Após a refatoração, o arquivo limpo não tem nenhum problema (Figura 2).

```
Windows PowerShell — test-smelly-fork
PS C:\...\test-smelly-fork> npx eslint test/userService.clean.test.js
PS C:\...\test-smelly-fork> echo "exit code: $LASTEXITCODE"
exit code: 0
PS C:\...\test-smelly-fork> █
```

Figura 2 – Lint do arquivo clean após a refatoração: o comando termina sem mensagens.

4. Validação

A suíte completa continua passando: os 14 testes do arquivo limpo e os 4 do arquivo original (o test.skip do original segue pulado, pois o arquivo não pode ser modificado).

```
Windows PowerShell — test-smelly-fork
PS C:\...\test-smelly-fork> npm test
> test-smells-lab@1.0.0 test
> jest

PASS test/userService.clean.test.js
PASS test/userService.smelly.test.js

Test Suites: 2 passed, 2 total
Tests:      1 skipped, 18 passed, 19 total
Snapshots:  0 total
Time:       0.608 s, estimated 1 s
Ran all test suites.
PS C:\...\test-smelly-fork> █
```

Figura 3 – Execução dos testes: 2 suítes aprovadas, 18 testes passando e 1 pulado.

Para provar a diferença de eficácia, a validação de idade foi removida temporariamente de src/userService.js (e restaurada em seguida). O teste original continuou verde, enquanto o teste refatorado falhou e apontou o problema.

```
Windows PowerShell — test-smelly-fork
# src/userService.js: validação "idade < 18" removida temporariamente
PS C:\...\test-smelly-fork> npx jest -t "menor de idade"
FAIL test/userService.clean.test.js
  ● UserService - Suíte de Testes Limpa > createUser > deve lançar erro ao criar usuário menor de idade

    expect(received).toThrow(expected)

    Expected substring: "O usuário deve ser maior de idade."

    Received function did not throw

      36 |         userService.createUser('Menor', 'menor@email.com', 17);
      37 |
    > 38 |         expect(criarMenorDeIdade).toThrow('O usuário deve ser maior de idade.');
```

Figura 4 – Sem a validação de idade, o teste smelly passa (falso positivo) e o teste clean falha.

5. Conclusão

A análise dos *test smells* mostrou que ter testes passando ou alta cobertura não significa, necessariamente, que eles sejam eficazes. Na suíte original, todos os testes estavam verdes, mas o teste de menor de idade continuou passando mesmo com a validação removida (Figura 4), porque o expect ficava dentro de um catch que nunca era executado. É o mesmo tipo de falha que o teste de mutação revela: um teste que não detecta o defeito.

A refatoração tornou os testes mais simples, claros e confiáveis. Removi a lógica condicional, troquei o try/catch por toThrow, dividi o Eager Test em testes focados, apliquei o padrão AAA, dei nomes descritivos e substituí a comparação com a string exata do relatório por verificações de conteúdo, menos frágeis a mudanças de formatação.

O ESLint com eslint-plugin-jest identificou automaticamente 6 problemas (4 erros e 2 avisos), mas não detectou o Teste Frágil nem o Eager Test, que só apareceram na análise manual. Assim, o trabalho reforçou a importância de combinar análise estática, revisão de código e testes bem estruturados, tratando o código de teste com o mesmo cuidado do código de produção, para ter uma suíte realmente confiável e um software sustentável.